

Setor de Educação Profissional e Tecnológica

Tecnologia em Análise e Desenvolvimento de Sistemas

DS122 - Desenvolvimento de Aplicações Web 1

07 - JavaScript: DOM e eventos

Prof. Alex Kutzke

Outubro de 2026

Objetivos da aula

Ao final desta aula você deve ser capaz de:

1. Descrever o DOM e distinguir o arquivo HTML da página aberta;
2. Selecionar elementos com `querySelector` e `querySelectorAll`;
3. Alterar texto, atributos e classes de um elemento;
4. Montar o catálogo na tela a partir do array de produtos;
5. Explicar o risco de `innerHTML` com dados do usuário;
6. Registrar ouvintes de evento com `addEventListener`;
7. Ligar a busca e o filtro da aula anterior aos campos da página;
8. Validar um formulário e dar o retorno na própria página.

Roteiro

Árvore

Seleção

Alteração

Criação

Eventos

Formulários

A página alterada pelo console

catalogo.html aberto, F12, aba **Console**:

```
document.querySelector("h1").textContent = "Loja Alterada";  
document.querySelector(".produto").remove();  
document.querySelectorAll(".produto").length    // 5
```

Título trocado, um cartão a menos, nenhum arquivo salvo.

Recarregue: tudo volta ao que estava.

DOM

Document Object Model, modelo de objetos do documento.

O navegador lê o HTML e cria um objeto para cada elemento, com:

- propriedades: texto, atributos, classes;
- métodos: acrescentar filho, remover-se, registrar evento.

A tela é desenhada a partir desses objetos, e redesenhada quando um deles muda.

Especificação do WHATWG, o mesmo grupo do HTML. Igual em todos os navegadores atuais.

Uma árvore de nós

```
document
  html
    head
      meta
      meta
      title
      link
    body
      header
        h1
        nav
      main
        section
          h2
          div.grade
            article.produto
            article.produto
```

Cada item é um **nó** (*node*).

Filho é o que está dentro. O texto de um elemento também é um nó.

É a mesma árvore que o seletor `.produto h3` percorria na aula de CSS.

O arquivo e a página aberta

Ctrl+U	o arquivo como chegou do servidor. Não muda.
aba Elements	o DOM naquele instante. Mostra cada alteração feita pelo JavaScript.

Quando o catálogo for montado pelo JavaScript, os cartões vão estar na tela e na aba **Elements**, e não no código-fonte.

Prática: o ambiente

No seu *fork* de `ds122-dom-assignment`:

1. copie suas páginas, `css/`, `imagens/` e `js/produtos.js`, ou mova o conteúdo de `base/` para a raiz;
2. crie `js/app.js` com `formataPreco`, `buscaPorNome` e `filtraPorFaixa` da tarefa anterior;
3. ligue `produtos.js` e `app.js`, nesta ordem, com `defer`;
4. no console, confira `produtos.length`.

Tempo: 5 minutos.

Resposta: o ambiente

```
<link rel="stylesheet" href="css/estilo.css">  
<script src="js/produtos.js" defer></script>  
<script src="js/app.js" defer></script>
```

```
produtos.length    // 6, com a pasta base/
```

Sem erro vermelho no console. Com `app.js` antes de `produtos.js`, o `app.js` não enxerga o array.

querySelector

O **primeiro** elemento que casa com um seletor CSS:

```
const titulo = document.querySelector("h1");  
const grade = document.querySelector(".grade");  
const campoNome = document.querySelector("#nome");  
const preco = document.querySelector(".produto .preco");
```

Também existe em cada elemento, e aí busca só dentro dele:

```
const cartao = document.querySelector(".produto");  
cartao.querySelector("h3");
```

querySelectorAll

Todos os que casam, numa NodeList:

```
const cartoes = document.querySelectorAll(".produto");
```

```
cartoes.length      // 6  
cartoes[0]          // o primeiro cartão  
cartoes.forEach((cartao) => console.log(cartao));
```

Tem `forEach`. Não tem `map`, `filter` nem `reduce`; para usá-los, `Array.from(cartoes)`.

```
cartoes.classList.add("destaque");  
// TypeError: Cannot read properties of undefined (reading  
↪ 'add')
```

getElementById

A forma anterior, só para id, e ainda em uso:

```
document.getElementById("nome");    // sem o #  
document.querySelector("#nome");    // o mesmo elemento
```

Nesta aula, `querySelector` em todos os casos.

Quando nada é encontrado

```
const botao = document.querySelector("#enviar"); // null
botao.addEventListener("click", enviar);
// TypeError: Cannot read properties of null
//           (reading 'addEventListener')
```

O erro aponta a segunda linha. A causa está na primeira:

- # ou . a menos no seletor;
- id escrito diferente do HTML;
- elemento que está em outra página;
- script sem defer.

Por que o script precisa de defer

Sem defer, o script executa quando o navegador encontra a tag, no `<head>`.

O `<body>` ainda não foi lido, e todo `querySelector` devolve `null`.

Com defer, o script espera o documento inteiro entrar na árvore.

Texto e atributos

```
const titulo = document.querySelector("main h2");
titulo.textContent = `Produtos (${produtos.length})`;

const imagem = document.querySelector(".produto img");
imagem.src = "imagens/cafe.jpg";
imagem.alt = "Pacote de café em grão de 500 g";

const campoNome = document.querySelector("#nome");
campoNome.value           // o que está digitado, sempre string
campoNome.value = "";     // apaga
```

textContent troca todo o conteúdo e não interpreta tags.

Classes: classList

```
const aviso = document.querySelector(".aviso");  
  
aviso.classList.add("oculto");  
aviso.classList.remove("oculto");  
aviso.classList.toggle("oculto");  
aviso.classList.contains("oculto")    // true ou false
```

```
.oculto {  
    display: none;  
}
```

Mexe só na classe informada.

class="produto" vira class="produto destaque".

style, e por que preferir classe

```
aviso.style.display = "none";  
cartao.style.borderColor = "gold";    // border-color
```

Funciona, com desvantagens em relação à classe:

- a aparência fica em dois lugares, CSS e JavaScript;
- desfazer exige saber o valor anterior;
- media query não alcança o que o script escreveu;
- a especificidade é a mais alta da cascata.

style para valor calculado na hora, como a largura de uma barra de progresso.

Prática: título e aviso

No `app.js`:

1. o `<h2>` da lista passa a mostrar `Produtos (6)`, com o número vindo de `produtos.length`;
2. a faixa `.aviso` some, por classe.

Depois compare o `<h2>` no `Ctrl+U` e na aba **Elements**.

Tempo: 6 minutos.

Resposta: título e aviso

```
const titulo = document.querySelector("main h2");
titulo.textContent = `Produtos (${produtos.length})`;
```

```
const aviso = document.querySelector(".aviso");
aviso.classList.add("oculto");
```

```
/* ----- 8. Classes ligadas pelo JavaScript ----- */
.oculto {
  display: none;
}
```

No Ctrl+U, o <h2> continua Produtos.

createElement e append

```
// 1. cria, fora da página
const aviso = document.createElement("p");

// 2. configura
aviso.textContent = "Frete grátis acima de R$ 100,00";
aviso.classList.add("destaque");

// 3. acrescenta como último filho
document.querySelector("main section").append(aviso);
```

append aceita vários nós, em ordem. Também: prepend, remove, e appendChild em código antigo.

Uma função que monta um cartão

```
const criaCartao = (produto) => {  
  const cartao = document.createElement("article");  
  cartao.classList.add("produto");  
  
  const nome = document.createElement("h3");  
  nome.textContent = produto.nome;  
  
  const preco = document.createElement("p");  
  preco.classList.add("preco");  
  preco.textContent = formataPreco(produto.preco);  
  
  cartao.append(nome, preco);  
  return cartao;  
};
```

Devolve o cartão; quem chama decide onde ele entra.

Prática: um cartão

1. acrescente o campo `imagem` aos objetos do `produtos.js` (em `base/` ele já existe);
2. escreva a `criaCartao` com `h3`, `img` com `alt`, `p.categoria` e `p.preco`;
3. acrescente à `.grade` o cartão de `produtos[0]`.

Resultado: um sétimo cartão, com o mesmo estilo dos outros.

Tempo: 8 minutos.

Resposta: um cartão

```
const criaCartao = (produto) => {  
  const cartao = document.createElement("article");  
  cartao.classList.add("produto");  
  const nome = document.createElement("h3");  
  nome.textContent = produto.nome;  
  const imagem = document.createElement("img");  
  imagem.src = produto.imagem;  
  imagem.alt = `Imagem de ${produto.nome}`;  
  const categoria = document.createElement("p");  
  categoria.classList.add("categoria");  
  categoria.textContent = produto.categoria;  
  const preco = document.createElement("p");  
  preco.classList.add("preco");  
  preco.textContent = formataPreco(produto.preco);  
  cartao.append(nome, imagem, categoria, preco);  
  return cartao;  
};  
  
const grade = document.querySelector(".grade");  
grade.append(criaCartao(produtos[0]));
```

De um array para a tela

```
const renderiza = (lista) => {  
  grade.replaceChildren();  
  lista.forEach((produto) => grade.append(criaCartao(produto)));  
};  
  
renderiza(produtos);
```

`replaceChildren()` esvazia a `grade`. Sem ele, a segunda chamada acrescenta cartões depois dos antigos. Em código antigo: `grade.innerHTML = .`

A lista vem por parâmetro: a mesma função vai mostrar o resultado da busca.

innerHTML

```
cartao.innerHTML = `  
  <h3>${produto.nome}</h3>  
  <p class="preco">${formataPreco(produto.preco)}</p>  
`;  
;
```

Mais curto, e interpreta como HTML tudo o que estiver no `\${}`.

Nome cadastrado com tags vira parte da página de todo visitante. Um `<img onerror="...">` executa JavaScript: *cross-site scripting*, XSS.

Na disciplina, `textContent` e `createElement` para dados. `innerHTML` só para trecho fixo, escrito no código.

Prática: o catálogo renderizado

1. apague do `catalogo.html` todos os `<article class="produto">`;
2. escreva a `renderiza` e chame `renderiza(produtos)`;
3. na `criaCartao`, desconto de 10% ou mais recebe a classe `destaque`;
4. escreva a regra `.destaque` no CSS.

O destaque condicional é requisito da Entrega 2.

Tempo: 10 minutos.

Resposta: o catálogo renderizado

```
<div class="grade"></div>
```

```
// dentro da criaCartao, antes do return  
if (produto.desconto >= 0.1) {  
  cartao.classList.add("destaque");  
}
```

```
.destaque {  
  border: 3px solid #b8860b;  
}
```

Na base/: café, mel e sabonete, com 15%, 10% e 20%.

addEventListener

```
const botao = document.querySelector("#mostrar-promocoes");

const mostraPromocoes = () => {
  renderiza(produtos.filter((produto) => produto.desconto > 0));
};

botao.addEventListener("click", mostraPromocoes);
```

Nome do evento e a função, chamada de ouvinte (*listener*).

A linha do `addEventListener` executa uma vez e só registra. A `mostraPromocoes` executa a cada clique.

Passar a função, não o resultado

```
botao.addEventListener("click", mostraPromocoes);    // certo
botao.addEventListener("click", mostraPromocoes());  // errado
```

Com parênteses, a função executa ao carregar a página, e o ouvinte registrado é o valor devolvido, `undefined`.

O efeito aparece uma vez sozinho, o clique não faz nada, e o console não mostra erro.

`onclick="..."` no HTML também funciona, e mistura comportamento com estrutura. Aqui os eventos ficam no script.

O objeto do evento

```
botao.addEventListener("click", (evento) => {  
  console.log(evento.type);    // "click"  
  console.log(evento.target);  // o elemento clicado  
});
```

O navegador passa o objeto ao ouvinte. O nome do parâmetro é livre: `event`, `evt` e `e` são comuns.

input e change

input	a cada alteração do valor: tecla, colagem, seta do campo numérico
change	quando a alteração é confirmada: ao sair do campo, ou ao escolher no <select>

```
const campoBusca = document.querySelector("#busca");

campoBusca.addEventListener("input", () => {
  renderiza(buscaPorNome(produtos, campoBusca.value));
});
```

buscaPorNome é a da aula anterior, sem alteração.

Prática: busca enquanto digita

1. acima da grade, um formulário com um campo `type="search", id="busca",` e o seu `<label>`;
2. ouvinte de `input` que chama a `renderiza` com o resultado da `buscaPorNome`.

Teste com `de`, e depois apague o termo.

Tempo: 8 minutos.

Resposta: busca enquanto digita

```
<form class="filtros" role="search">
  <p>
    <label for="busca">Buscar por nome</label>
    <input type="search" id="busca">
  </p>
</form>
```

Com de: chá de hibisco, farinha de milho, sabonete de argila, cesta de vime.

Campo vazio: `includes()` é true para qualquer nome, e todos voltam.

value é sempre string

<code>campoMax.value</code>	<code>// "50", mesmo em type="number"</code>
<code>campoMax.value + 1</code>	<code>// "501"</code>
<code>Number(campoMax.value)</code>	<code>// 50</code>
<code>Number("")</code>	<code>// 0</code>

Converta antes de fazer conta.

Preço máximo vazio convertido direto vira 0, e nenhum produto passa.

Prática: faixa de preço

1. dois campos `type="number"`, `preco-min` e `preco-max`, com `<label>`;
2. uma função `aplicaFiltros` que lê os três campos e aplica `buscaPorNome` e `filtraPorFaixa`;
3. a mesma função como ouvinte de `input` nos três campos;
4. campo vazio não filtra nada.

Teste: de na busca, faixa de 10 a 30.

Tempo: 10 minutos.

Resposta: faixa de preço

```
const campoMin = document.querySelector("#preco-min");
const campoMax = document.querySelector("#preco-max");

const aplicaFiltros = () => {
  const minimo = campoMin.value === "" ? 0 : Number(campoMin.value);
  const maximo = campoMax.value === "" ? Infinity : Number(campoMax.value);

  const porNome = buscaPorNome(produtos, campoBusca.value);
  renderiza(filtrarPorFaixa(porNome, minimo, maximo));
};

campoBusca.addEventListener("input", aplicaFiltros);
campoMin.addEventListener("input", aplicaFiltros);
campoMax.addEventListener("input", aplicaFiltros);
```

Infinity é maior que qualquer número. Resultado do teste: chá de hibisco e farinha de milho.

submit e preventDefault

Envio do formulário: evento submit no <form>, depois a requisição para o action, que carrega outra página.

```
const formulario = document.querySelector("form");

formulario.addEventListener("submit", (evento) => {
  evento.preventDefault();  // cancela o envio
});
```

Ouvinte no <form> com submit, e não no botão com click: o submit só acontece quando o formulário vai mesmo ser enviado.

Sem PHP ainda, o contato cancela sempre e responde na própria página.

Validação nativa e em JavaScript

A nativa (required, minlength, type) executa **antes** do submit. Campo obrigatório vazio: o ouvinte nem é chamado.

O JavaScript cobre o que os atributos não expressam:

```
"    ".length           // 3: passa por minlength="3"
"    ".trim().length     // 0

if (campoNome.value.trim().length < 3) {
    // recusa
}
```

Nenhuma das duas protege o servidor. Essa validação vem com o PHP.

Mensagem de erro no lugar do campo

```
<input type="text" id="nome" name="nome" minlength="3" required>  
<span class="erro" id="erro-nome"></span>
```

```
formulario.addEventListener("submit", (evento) => {  
  evento.preventDefault();  
  erroNome.textContent = "";      // apaga a anterior  
  
  if (campoNome.value.trim().length < 3) {  
    erroNome.textContent = "Informe o nome, com pelo menos 3 letras.";  
  }  
});
```

Perto do campo, e não num alert, que bloqueia a página e some ao fechar.

Prática: validação do contato

Em `contato.html`, com `js/contato.js` ligado:

1. `preventDefault` no `submit`;
2. nome com menos de 3 caracteres depois do `trim`: mensagem abaixo do nome;
3. mensagem com menos de 10: mensagem abaixo dela;
4. tudo certo: Mensagem enviada com sucesso! na página, e o formulário limpo.

Teste: três espaços no nome e oi na mensagem.

Tempo: 12 minutos.

Resposta: validação do contato

```
formulario.addEventListener("submit", (evento) => {  
  evento.preventDefault();  
  erroNome.textContent = "";  
  erroMensagem.textContent = "";  
  sucesso.classList.add("oculto");  
  let valido = true;  
  
  if (campoNome.value.trim().length < 3) {  
    erroNome.textContent = "Informe o nome, com pelo menos 3 letras.";  
    valido = false;  
  }  
  if (campoMensagem.value.trim().length < 10) {  
    erroMensagem.textContent = "A mensagem precisa ter pelo menos 10 caracteres.";  
    valido = false;  
  }  
  if (valido) {  
    sucesso.classList.remove("oculto");  
    formulario.reset();  
  }  
});
```

Quando dá errado

Sintoma	Causa provável
Cannot read properties of null	seletor sem resultado, ou script sem defer
(reading 'add') numa lista	classList direto na NodeList
clique não faz nada, efeito ao abrir	função chamada no addEventListener
soma dá "501"	value sem Number
cartões repetidos a cada filtro	contêiner não esvaziado
página recarrega no envio	preventDefault ausente

Os quatro últimos não aparecem no console. `console.log` dentro do ouvinte confirma se ele foi chamado, e com que valores.

Tarefa

Os exercícios valem nota e compõem o item *Exercícios em sala*.

Enunciado no README.md de ds122-dom-assignment; prazo na UFPR Virtual.

Entrega pelo *fork* no GitLab, individual ou em dupla, sem uso de IA generativa para produzir o código.

Busca, faixa de preço, destaque e validação são requisitos da Entrega 2. O que falta, carregar os produtos de um JSON com `fetch`, é o assunto do próximo encontro.